Ejercicio de Aplicación 🤓

L4: Algoritmos de clusterización

Es hora de que pongas en práctica todo lo aprendido. 🤓

Este apartado tiene el objetivo de ayudarte a seguir potenciando tus habilidades, por lo que a continuación encontrarás diferentes **desafíos** que podrás resolver de forma independiente y a tu ritmo.

Más adelante conseguirás las resoluciones para que valides tus respuestas y puedas monitorear tu progreso. 😉

¡Manos a la obra!

1. **Desafío 🎯**

   Realizar un análisis comparativo práctico de los algoritmos de clusterización: K-Means, Agrupamiento Jerárquico y DBSCAN.

   Pasos

   - *Cargar un dataset de datos sin etiquetar (puedes usar iris de scikit-learn pero sin la variable de clase).*
   - *Aplicar los tres algoritmos: K-Means, Agrupamiento Jerárquico y DBSCAN.*
   - *Generar visualizaciones en 2D para cada uno.*
   - *Explicar qué observas en cada caso.*
   - *Comentar cuál método se adapta mejor a los datos y por qué.*

2. **¿Dónde se lleva a cabo? 👩‍💻**

   Visual Studio Code

3. **Tiempo de dedicación ⏳**

   1 Hora.

4. **Recursos🛠️**

   - *Dataset Iris (from sklearn.datasets import load_iris)*
   - *Bibliotecas: scikit-learn, matplotlib, pandas, scipy*

5. **Plus ➕**

   Puedes agregar una métrica de validación como el coeficiente de silueta para comparar los resultados obtenidos por cada algoritmo.

6. **⚠️Condición**

   Esta práctica o ejercitación **no requiere ser entregada y/o evaluada** por el mentor. No obstante puedes compartir tus resultados con el resto de los bootcampers y construir conocimiento en conjunto.

Ejercicio de Aplicación 🤓

L4: Algoritmos de clusterización

Resolución del desafío 👇

```python
# Cargar dataset Iris sin la variable de clase
from sklearn.datasets import load_iris
import pandas as pd

data = load_iris()
X = pd.DataFrame(data.data, columns=data.feature_names)

# K-Means
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

kmeans = KMeans(n_clusters=3, random_state=0)
X['kmeans_cluster'] = kmeans.fit_predict(X)

plt.scatter(X.iloc[:, 0], X.iloc[:, 1], c=X['kmeans_cluster'])
plt.title("K-Means Clustering")
plt.show()

# Agrupamiento Jerárquico
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.cluster import AgglomerativeClustering

linked = linkage(X.iloc[:, :-1], 'ward')
dendrogram(linked)
plt.title("Dendrograma - Agrupamiento Jerárquico")
plt.show()

agglo = AgglomerativeClustering(n_clusters=3)
X['agglo_cluster'] = agglo.fit_predict(X.iloc[:, :-2])

plt.scatter(X.iloc[:, 0], X.iloc[:, 1], c=X['agglo_cluster'])
plt.title("Clustering Jerárquico")
plt.show()
```

```python
# DBSCAN
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler

X_scaled = StandardScaler().fit_transform(X.iloc[:, :-3])
dbscan = DBSCAN(eps=0.5, min_samples=5)
X['dbscan_cluster'] = dbscan.fit_predict(X_scaled)

plt.scatter(X.iloc[:, 0], X.iloc[:, 1], c=X['dbscan_cluster'])
plt.title("DBSCAN Clustering")
plt.show()
```